EXPERIMENT 13 – SEQUENTIAL DATASET PREPARATION AND RNN PREDICTION

1. Title

Sequential Dataset Preparation and RNN Prediction

2. Aim

To prepare time-series data for sequential learning by handling missing values, scaling the data and creating sliding windows. To build and train a SimpleRNN model for next-hour electricity demand prediction and evaluate its performance using MAE and RMSE.

3. Mathematical Formulae

1. Min-Max Normalisation

$$X_{scaled} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

2. Sliding Window

For a sequence of 24 past observations:

$$X_t = [x_{t-24}, x_{t-23}, ..., x_{t-1}]$$

$$Y_t = x_t$$

3. SimpleRNN Hidden State

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

Where:

- x_t = Input at time step t
- h_t = Current hidden state
- h_{t-1} = Previous hidden state
- W_{xh} = Input-to-hidden weights
- W_{hh} = Recurrent weights
- b_h = Bias

4. Output Prediction

$$\hat{y}_t = W_{hy} h_t + b_y$$

5. Mean Absolute Error (MAE)

$$MAE = \frac{1}{n} \sum_{i=1}^{n} |y_i - \hat{y}_i|$$

6. Root Mean Squared Error (RMSE)

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2}$$

4. System Architecture



5. Algorithm

1. Import the required libraries such as TensorFlow, Keras, NumPy, Pandas and Matplotlib.
2. Generate or load hourly electricity demand data using Pandas.
3. Handle missing values using interpolation.
4. Scale the demand values using Min-Max normalisation.
5. Create sliding windows using the previous 24 hours to predict the next hour.
6. Split the sequences chronologically into training and testing sets.
7. Build a SimpleRNN model using Keras.
8. Compile the model using Adam optimiser and mean squared error loss.
9. Train the model using the training sequences.
10. Predict electricity demand using the test sequences.
11. Calculate MAE and RMSE.
12. Plot actual demand against predicted demand.

6. Python Program

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import tensorflow as tf


from sklearn.preprocessing import MinMaxScaler

from sklearn.metrics import mean_absolute_error, mean_squared_error


# 1. Generate hourly electricity demand data

np.random.seed(42)


hours = 365 * 24


dates = pd.date_range(

   start="2025-01-01",
```

```
    periods=hours,
    freq="h"
)

daily_pattern = 20 * np.sin(
    2 * np.pi * np.arange(hours) / 24
)

weekly_pattern = 8 * np.sin(
    2 * np.pi * np.arange(hours) / (24 * 7)
)

seasonal_pattern = 15 * np.sin(
    2 * np.pi * np.arange(hours) / (24 * 365)
)

noise = np.random.normal(0, 4, hours)

demand = (
    100
    + daily_pattern
    + weekly_pattern
    + seasonal_pattern
    + noise
)

df = pd.DataFrame({
    "Date": dates,
    "Demand": demand
})
```

```
# 2. Handle missing values
df.loc[10:12, "Demand"] = np.nan

df["Demand"] = df["Demand"].interpolate()

print("Dataset:")
print(df.head())

# 3. Scale the data
scaler = MinMaxScaler()

scaled_data = scaler.fit_transform(
  df[["Demand"]]
)

# 4. Create sliding windows
window_size = 24

X = []
y = []

for i in range(window_size, len(scaled_data)):

  X.append(
    scaled_data[i-window_size:i]
  )

  y.append(
    scaled_data[i]
  )
```

```
X = np.array(X)
y = np.array(y)

print("Input shape:", X.shape)
print("Output shape:", y.shape)

# 5. Chronological train-test split
split = int(len(X) * 0.80)

X_train = X[:split]
X_test = X[split:]

y_train = y[:split]
y_test = y[split:]

# 6. Build SimpleRNN model
model = tf.keras.Sequential([

  tf.keras.layers.Input(
    shape=(window_size, 1)
  ),

  tf.keras.layers.SimpleRNN(
    50,
    activation="tanh"
  ),

  tf.keras.layers.Dense(1)

])
```

```
# 7. Compile model
model.compile(
    optimizer="adam",
    loss="mse",
    metrics=["mae"]
)

model.summary()

# 8. Train model
history = model.fit(
    X_train,
    y_train,
    epochs=15,
    batch_size=32,
    validation_split=0.1,
    verbose=1
)

# 9. Predict demand
predictions = model.predict(X_test)

# 10. Convert scaled values back to original values
y_test_actual = scaler.inverse_transform(y_test)

predictions_actual = scaler.inverse_transform(
    predictions
)

# 11. Calculate performance metrics
mae = mean_absolute_error(
```

```
    y_test_actual,
    predictions_actual
)

rmse = np.sqrt(
    mean_squared_error(
        y_test_actual,
        predictions_actual
    )
)

print("\nPerformance Metrics")

print("MAE:", mae)
print("RMSE:", rmse)

# 12. Plot actual vs predicted demand
plt.figure(figsize=(12, 5))

plt.plot(
    y_test_actual[:200],
    label="Actual Demand"
)

plt.plot(
    predictions_actual[:200],
    label="Predicted Demand"
)

plt.title("Actual vs Predicted Electricity Demand")
```

```
plt.xlabel("Time Steps")
plt.ylabel("Electricity Demand")

plt.legend()
plt.show()

# 13. Plot training and validation loss
plt.figure(figsize=(8, 5))

plt.plot(
    history.history["loss"],
    label="Training Loss"
)

plt.plot(
    history.history["val_loss"],
    label="Validation Loss"
)

plt.title("RNN Training and Validation Loss")

plt.xlabel("Epochs")
plt.ylabel("MSE Loss")

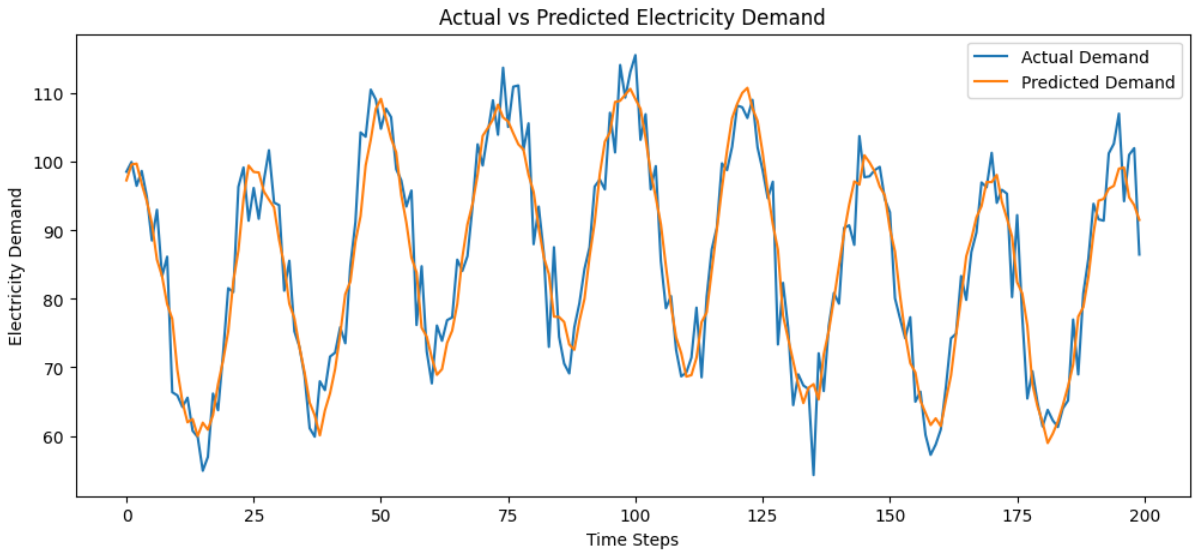
plt.legend()
plt.show()
```

7. Output

```
Dataset:
                 Date      Demand
0 2025-01-01 00:00:00  101.986857
1 2025-01-01 01:00:00  104.933212
2 2025-01-01 02:00:00  113.210113
3 2025-01-01 03:00:00  121.162247
4 2025-01-01 04:00:00  117.619268
Input shape: (8736, 24, 1)
Output shape: (8736, 1)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
simple_rnn (SimpleRNN)	(None, 50)	2,600
dense (Dense)	(None, 1)	51

```
Total params: 2,651 (10.36 KB)
Trainable params: 2,651 (10.36 KB)
Non-trainable params: 0 (0.00 B)
Performance Metrics
MAE: 3.933528356289811
RMSE: 4.896743276118979
```





8. Result

Thus, sequential time-series data was successfully prepared using sliding windows and chronological splitting. A SimpleRNN model was trained to predict next-hour electricity demand and evaluated using Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE).